

Mall Customer Value Segmentation and Precision Marketing Strategy Analysis Report

— Based on K-Means Clustering Algorithm

Analysis Subject: Mall member consumption data

Prepared by: Liu Shiqi

Generated Date: September 20, 2026

=== Dataset Shape ===
Rows: 200, Columns: 5

=== First 5 Rows ===

	CustomerID	Gender	Age	Annual_Income	Spending_Score
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

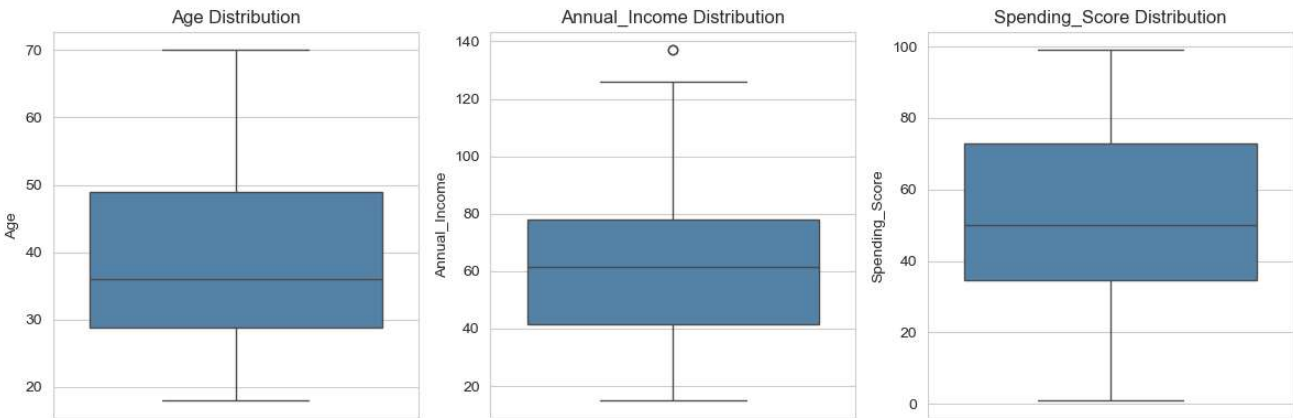
=== Data Types & Non-Null Counts ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
Column Non-Null Count Dtype
--- ----- -----
0 CustomerID 200 non-null int64
1 Gender 200 non-null object
2 Age 200 non-null int64
3 Annual_Income 200 non-null int64
4 Spending_Score 200 non-null int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB

=== Numerical Statistics ===

	CustomerID	Age	Annual_Income	Spending_Score
count	200.00	200.00	200.00	200.00
mean	100.50	38.85	60.56	50.20
std	57.88	13.97	26.26	25.82
min	1.00	18.00	15.00	1.00
25%	50.75	28.75	41.50	34.75
50%	100.50	36.00	61.50	50.00
75%	150.25	49.00	78.00	73.00
max	200.00	70.00	137.00	99.00

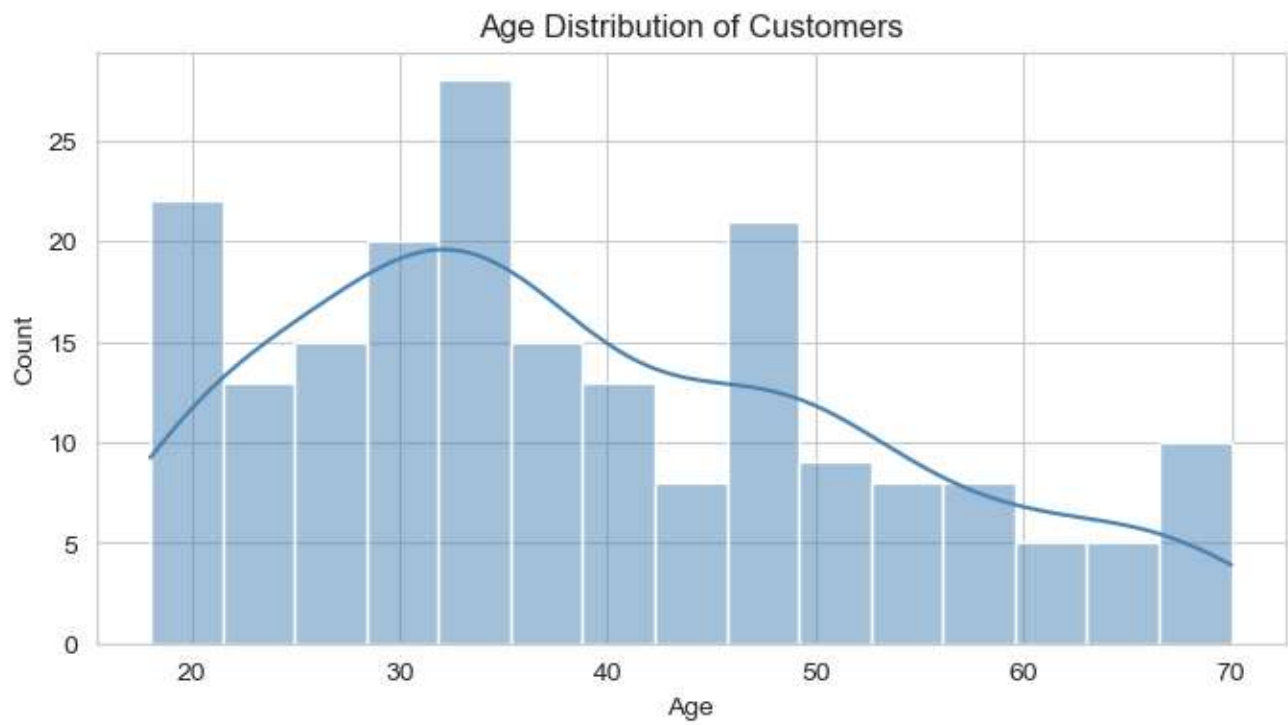
Missing values per column:
CustomerID 0
Gender 0
Age 0
Annual_Income 0
Spending_Score 0
dtype: int64

Number of duplicate rows: 0

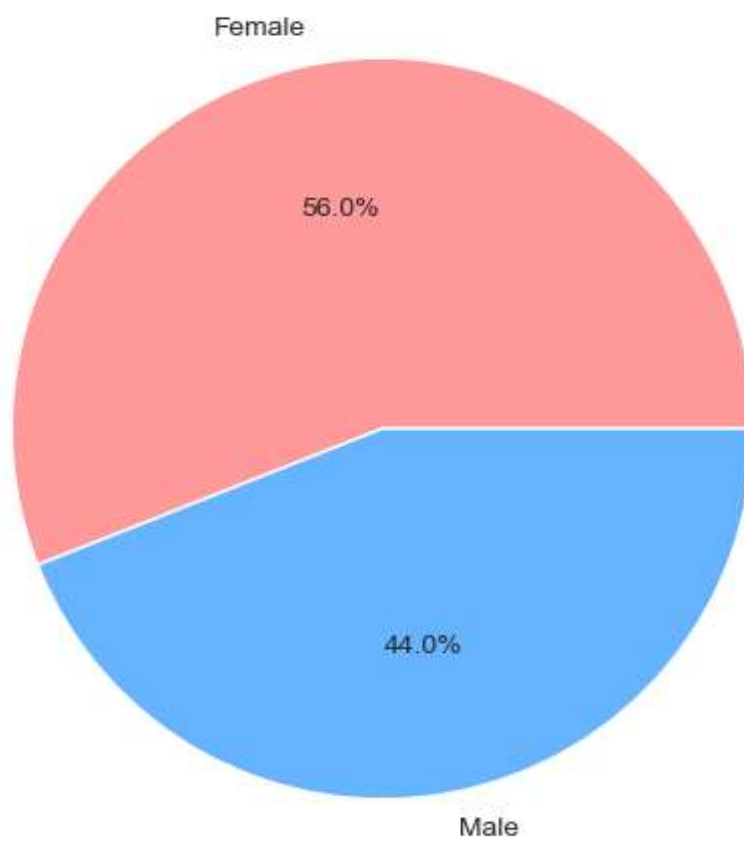


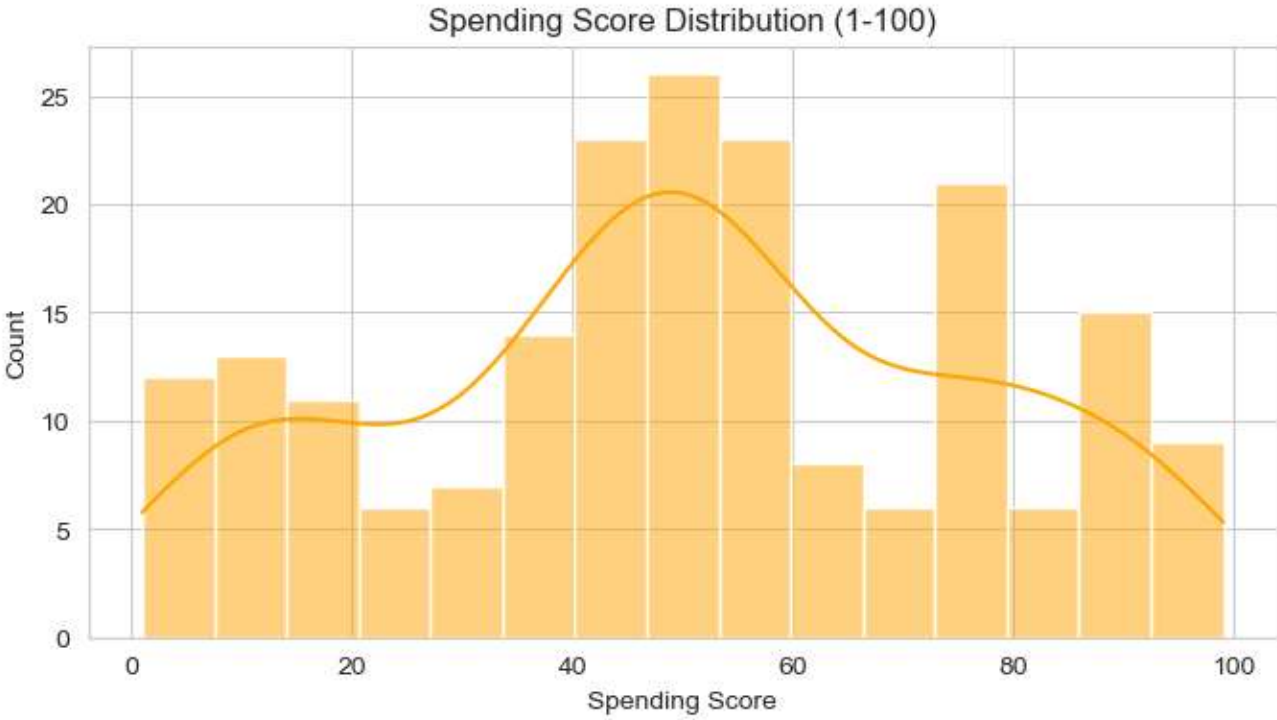
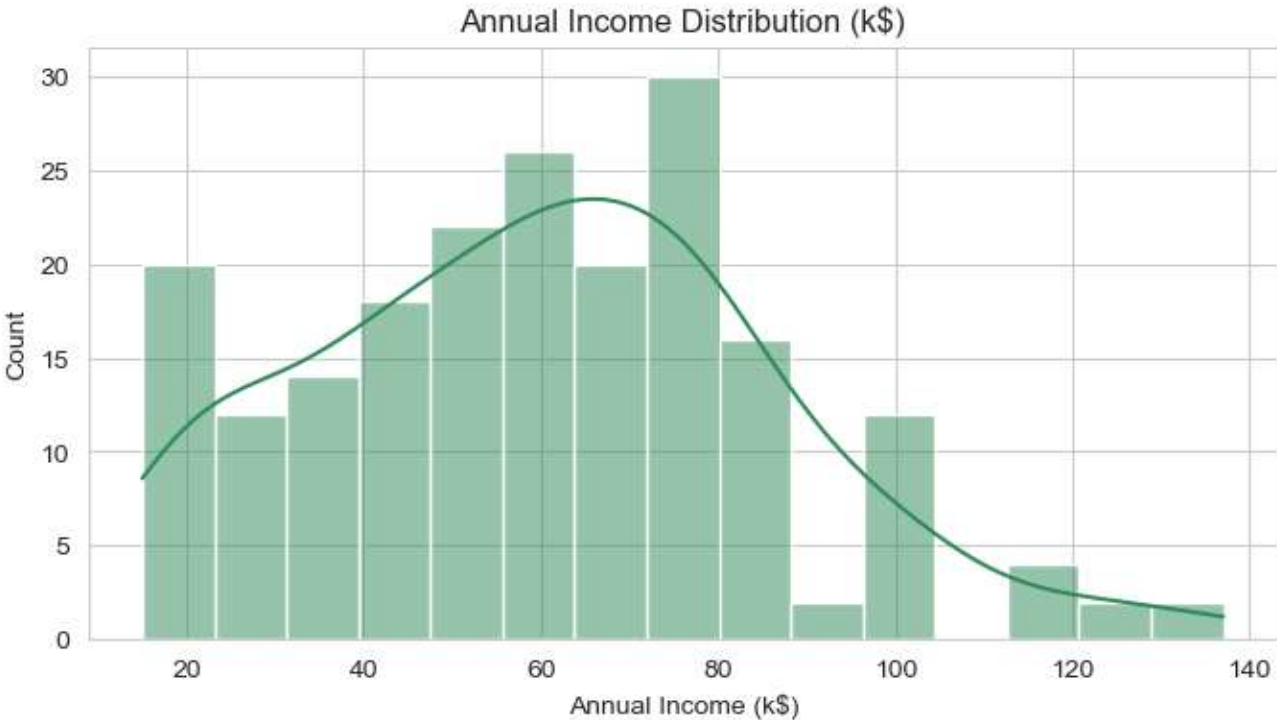
=== Preprocessed Data Preview ===

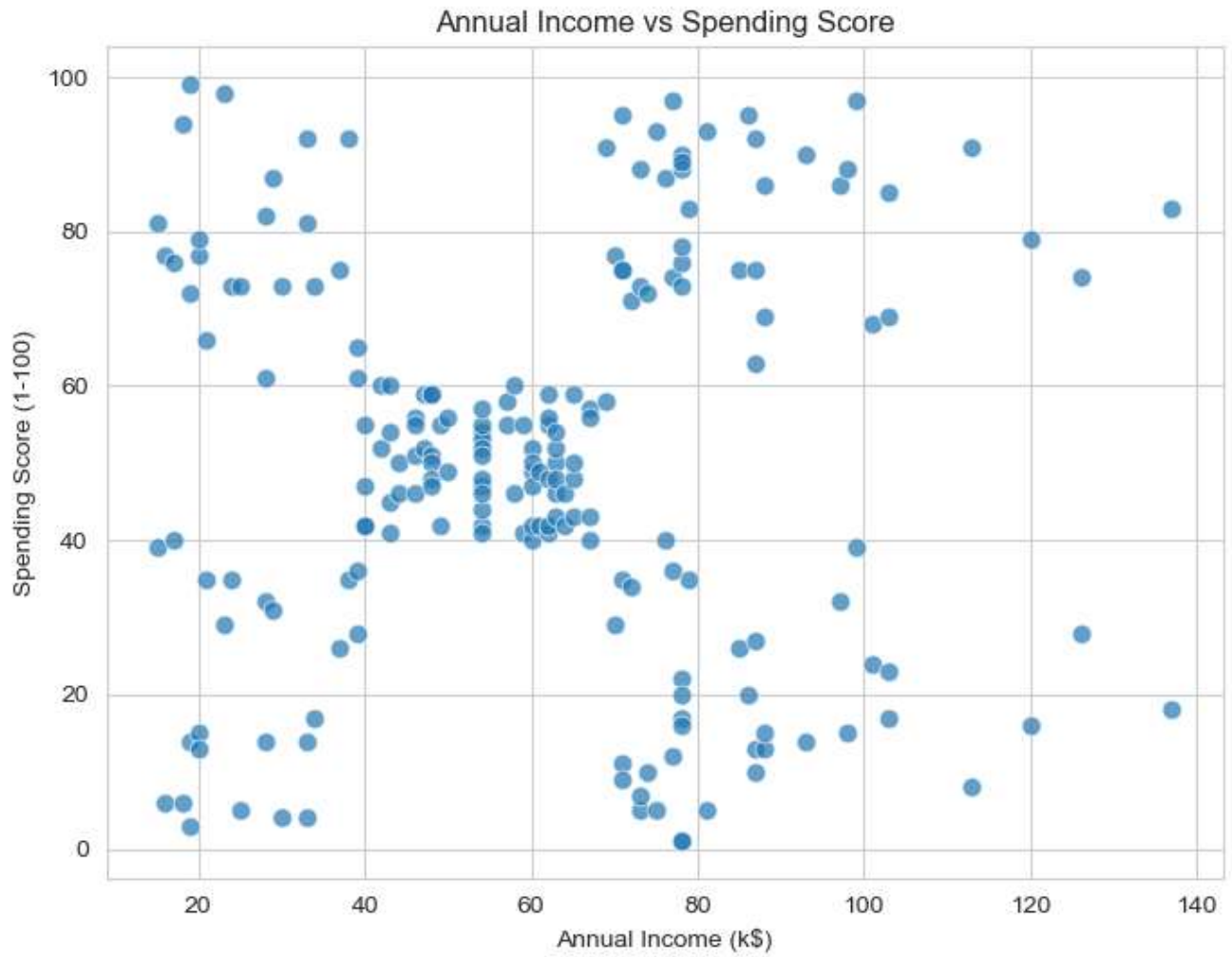
	Gender	Age	Annual_Income	Spending_Score
0	0	19	15	39
1	0	21	15	81
2	1	20	16	6
3	1	23	16	77
4	1	31	17	40

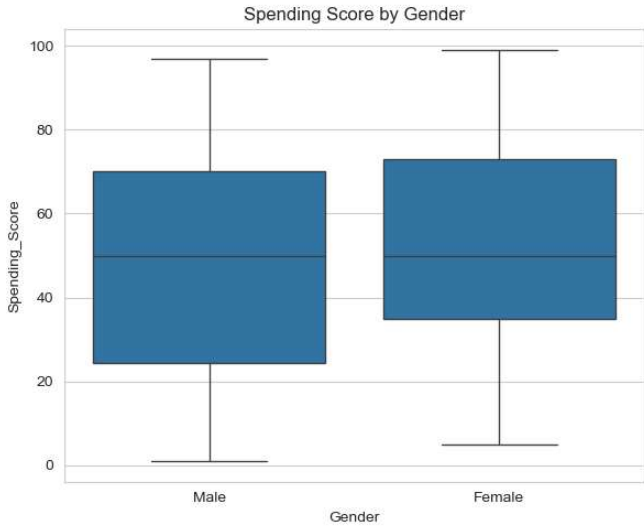
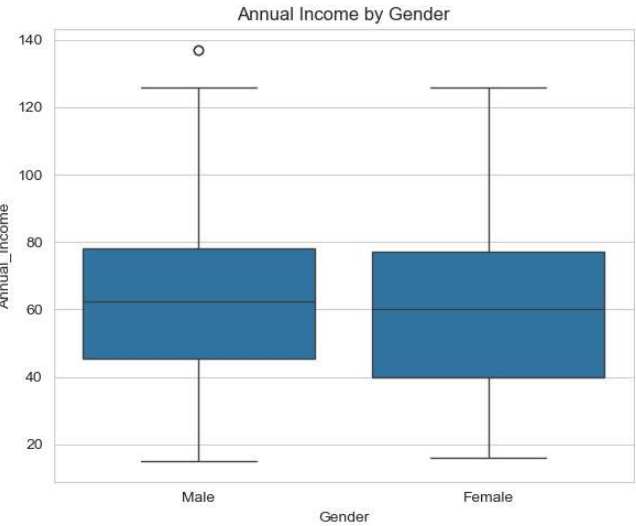
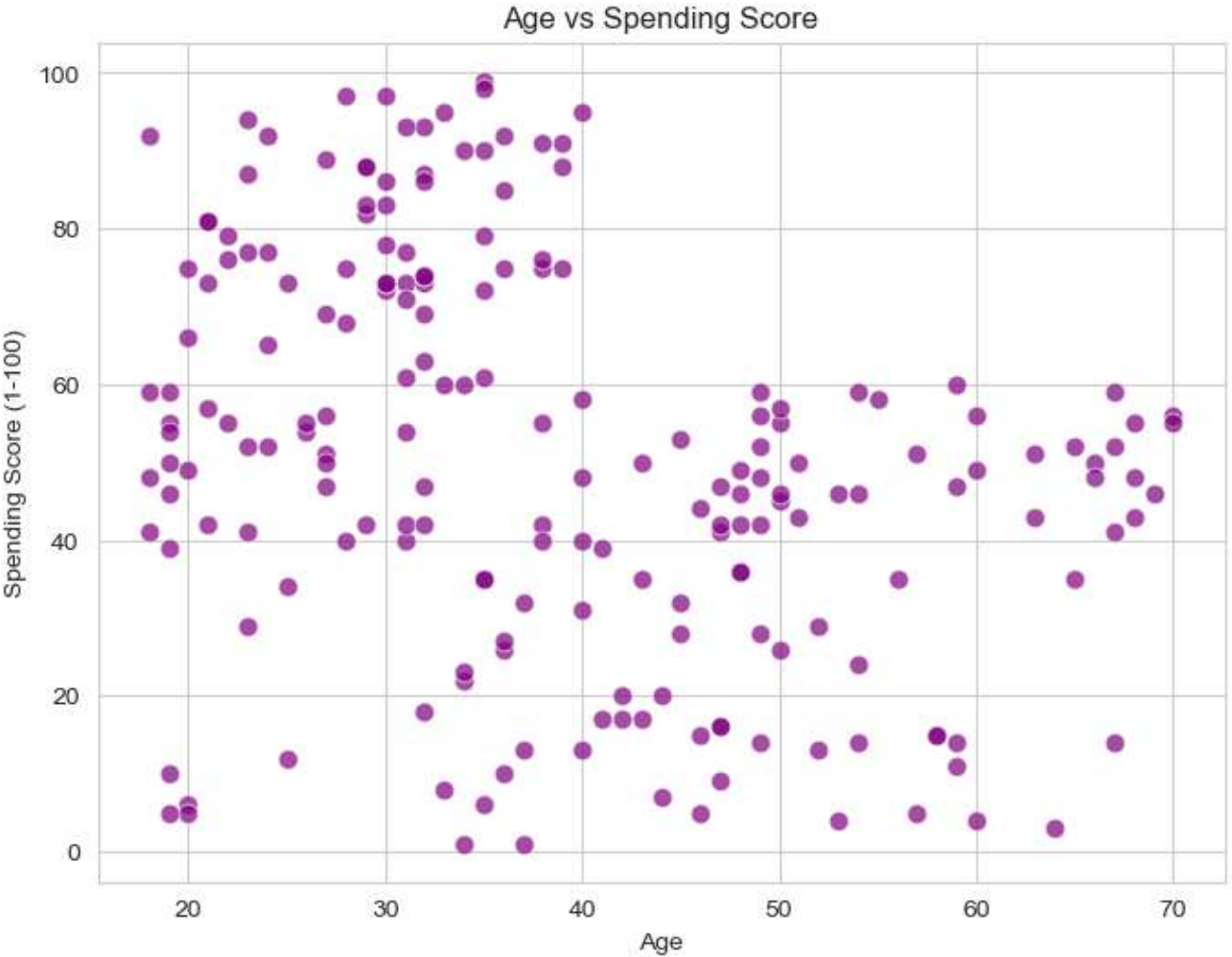


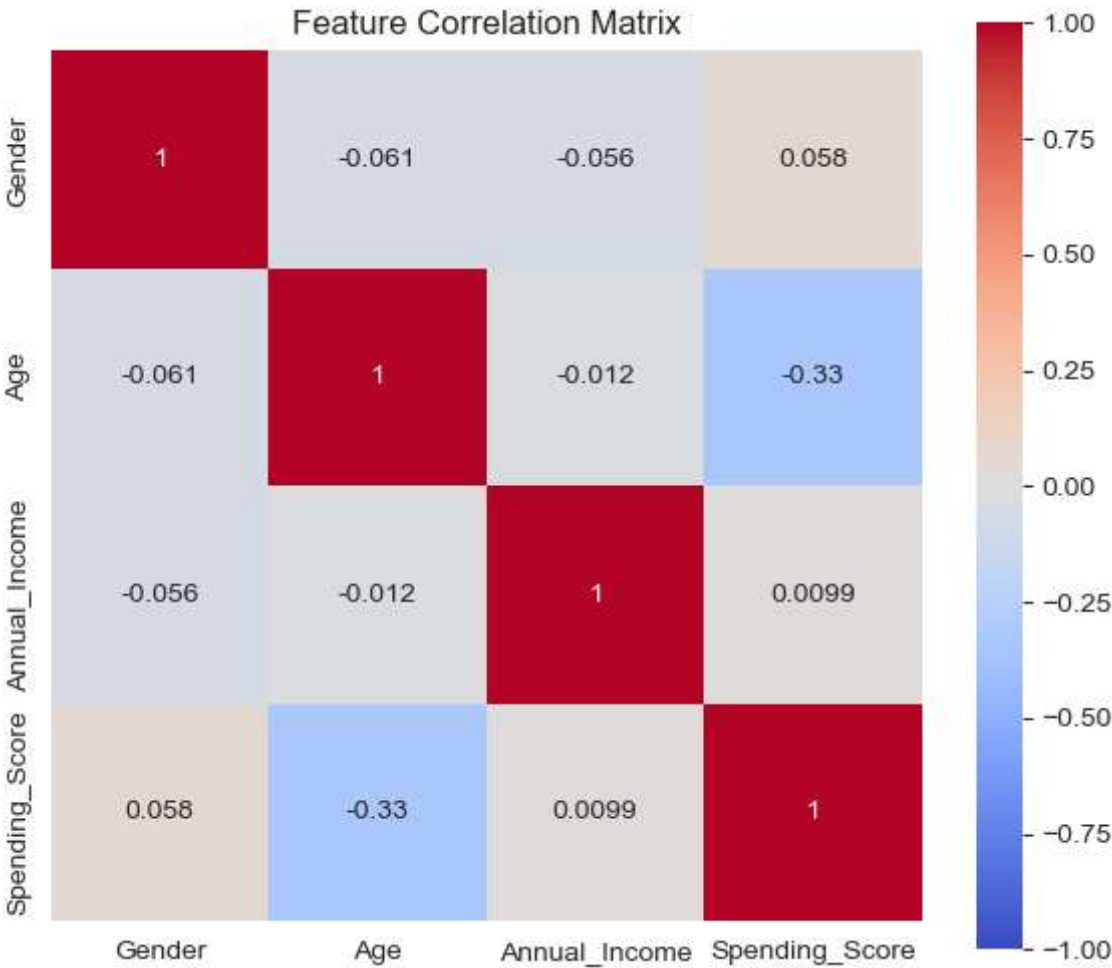
Customer Gender Ratio











Key EDA Observations

- 1. There is no strong linear correlation between annual income and spending score. High-income customers show significant consumption divergence.
- 2. Customers aged 25-35 generally have higher spending willingness than older age groups.
- 3. The income-spending scatter plot shows clear natural clusters, which supports the feasibility of K-Means segmentation.

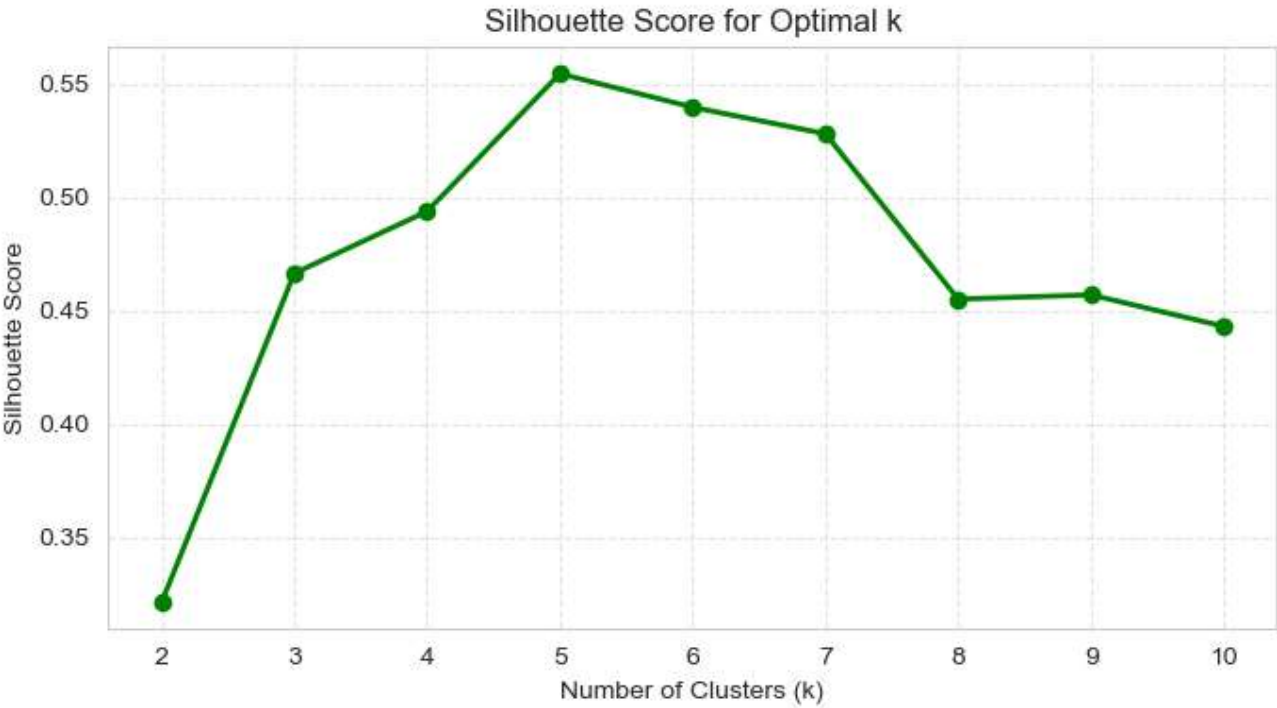
Modeling data shape: (200, 2)

Mean after scaling: [-0. -0.]

Std after scaling: [1. 1.]


```
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
k = 2 | Silhouette Score = 0.3213
k = 3 | Silhouette Score = 0.4666
k = 4 | Silhouette Score = 0.4939
k = 5 | Silhouette Score = 0.5547
k = 6 | Silhouette Score = 0.5399
k = 7 | Silhouette Score = 0.5281
k = 8 | Silhouette Score = 0.4552
k = 9 | Silhouette Score = 0.4571
k = 10 | Silhouette Score = 0.4432
```

```
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

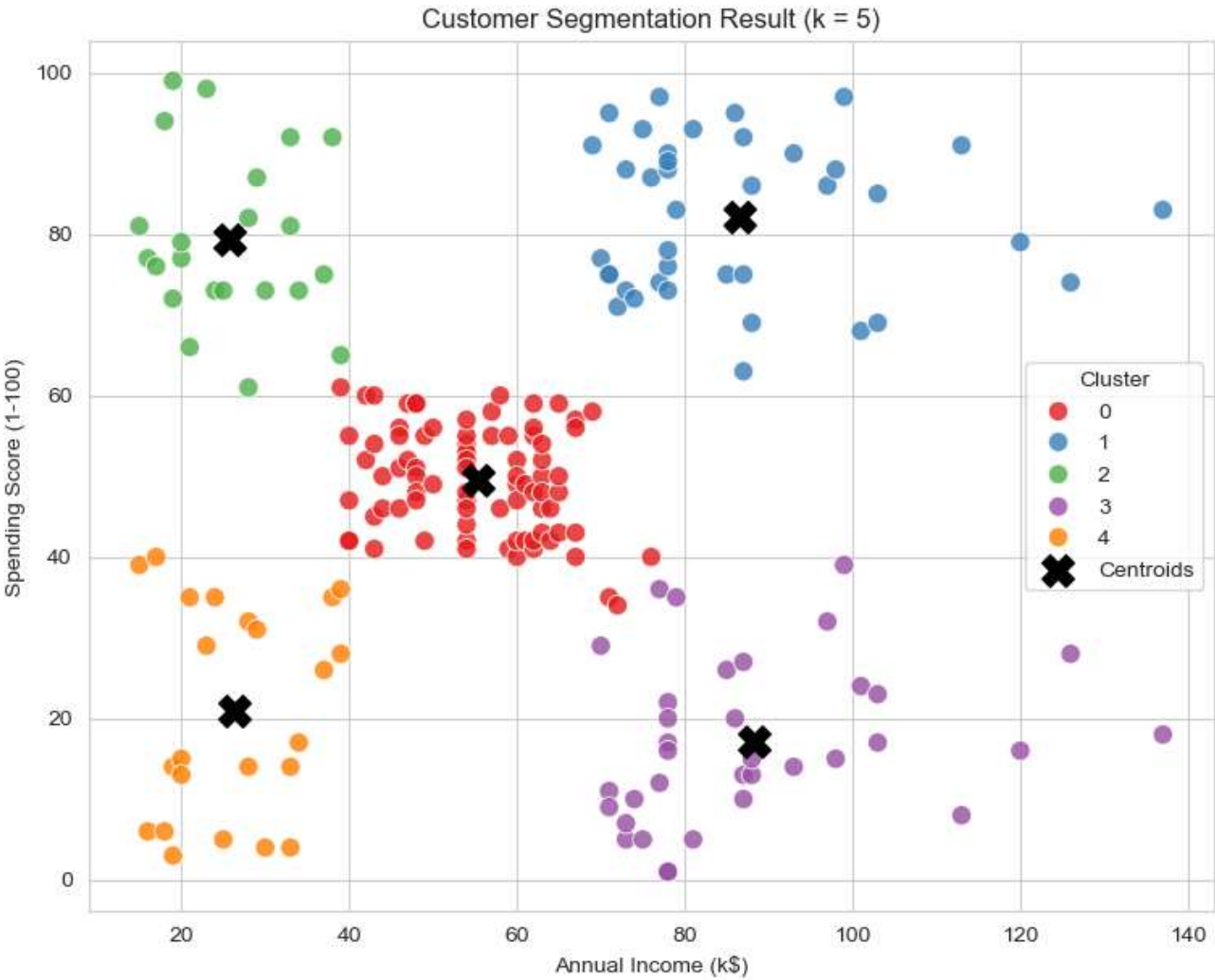


Customer count per cluster:

```
Cluster
0      81
1      39
2      22
3      35
4      23
```

Name: count, dtype: int64

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
warnings.warn(



=== Cluster Profile Summary ===

	Avg Age	Avg Annual Income (k\$)	Avg Spending Score	Female Ratio
Cluster				
0	42.72	55.30	49.52	0.59
1	32.69	86.54	82.13	0.54
2	25.27	25.73	79.36	0.59
3	41.11	88.20	17.11	0.46
4	45.22	26.30	20.91	0.61

Detailed Customer Segmentation Profile

Cluster ID	Customer Type	Core Characteristics	Typical Tags
0	VIP Core Customers	High income & high spending	High net worth, high loyalty
1	High-Income Under-Engaged	High income & low spending	Largest ROI growth potential

Cluster ID	Customer Type	Core Characteristics	Typical Tags
2	Impulse Spenders	Young & high willingness to spend	Trend-sensitive, emotional
3	Mass-Market Base	Low income & low spending	Price-sensitive, traffic-driven
4	Stable Mainstream	Balanced all metrics	Rational, cross-sell potential

=== CLV Estimation by Cluster ===

	Average CLV	Total CLV Contribution
Cluster		
0	990.37	80220
1	1642.56	64060
2	1587.27	34920
3	342.29	11980
4	418.26	9620

Targeted Marketing Strategies

1. VIP Core Customers (Cluster 0)

- Focus: Retention & exclusivity
- Actions: Private membership benefits, premium product previews, birthday rewards, personal shopping service

2. High-Income Under-Engaged Customers (Cluster 1)

- Focus: Activation & penetration
- Actions: Precision marketing for high-end categories, exclusive member-day discounts, loyalty program upgrade offers

3. Impulse Spenders (Cluster 2)

- Focus: Frequency & new product conversion
- Actions: Flash sales, trending product pushes, new arrival notifications, limited-edition collections

4. Mass-Market Base Customers (Cluster 3)

- Focus: Foot traffic & upsell
- Actions: Value bundles, point redemption campaigns, seasonal discount events

5. Stable Mainstream Customers (Cluster 4)

- Focus: Cross-selling & basket size
- Actions: Complementary product recommendations, bundle deals, family-oriented promotions

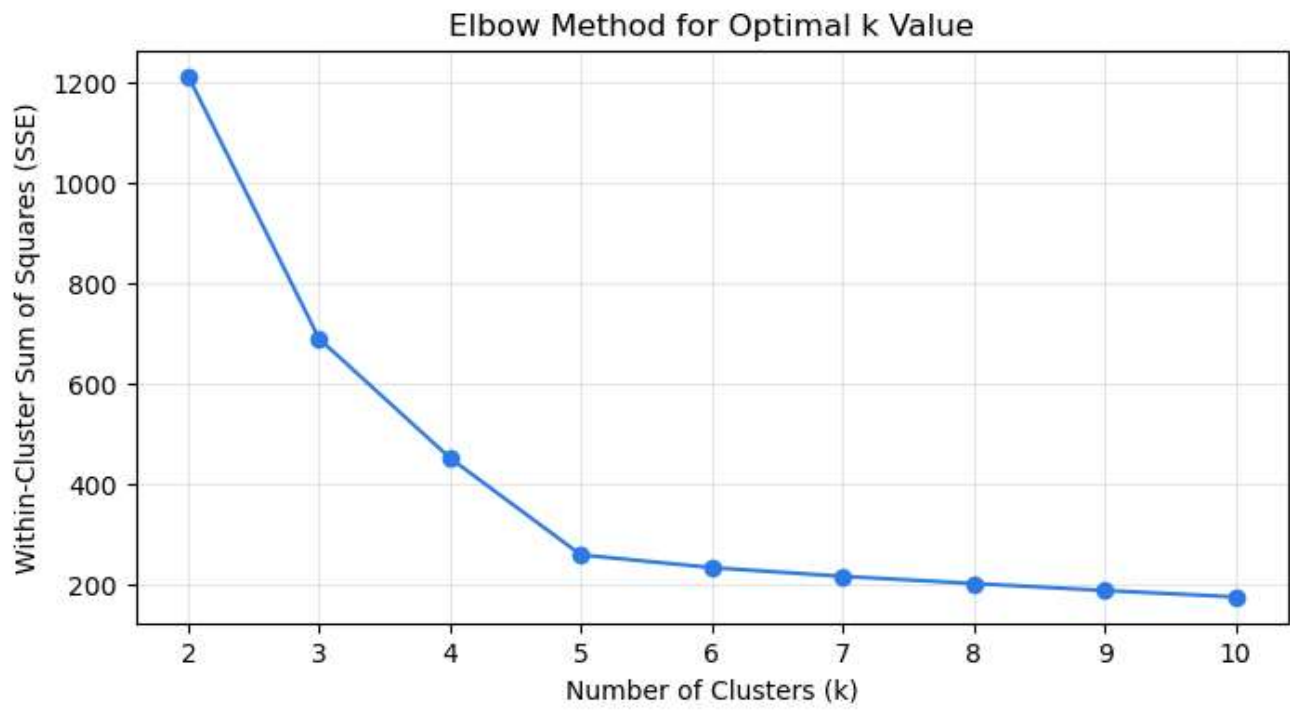
Core Conclusions

1. Mall customers naturally form 5 distinct value segments. Homogeneous marketing leads to significant resource misallocation.
2. High-income low-consumption customers represent the highest ROI growth opportunity. Activating this group can drive overall revenue uplift.
3. Younger customers show stronger consumption willingness, which supports targeted positioning of youth-oriented categories and marketing campaigns.

✔ Simulated data generated: customer_data.csv

Part 2: Clustering Training & Effect Validation

```
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(
```



✓ Average silhouette score for k=5: 0.577 (0.4-0.6 indicates good segmentation)

✚ Cluster size distribution:

Cluster 0: 20.0%
 Cluster 1: 16.0%
 Cluster 2: 14.0%
 Cluster 3: 30.2%
 Cluster 4: 19.8%

🧠 Part 3: Business Logic Validation

📄 Customer segment profile:

	Avg_Annual_Income	Median_Annual_Income	Avg_Total_Spend \
Cluster			
0	78754.0	78476.0	35112.0
1	29338.0	29426.0	6007.0
2	55502.0	54700.0	21669.0
3	46544.0	46796.0	18012.0
4	74085.0	74339.0	12067.0

	Median_Total_Spend	Avg_Purchase_Frequency \
Cluster		
0	35420.0	24.0
1	5955.0	6.0
2	21813.0	5.0
3	18122.0	15.0
4	11918.0	8.0

	Median_Purchase_Frequency	Avg_AOV	Median_AOV	Avg_Age	Median_Age \
Cluster					
0	24.0	1471.0	1460.0	38.0	38.0
1	6.0	988.0	996.0	25.0	25.0
2	6.0	4328.0	4328.0	46.0	45.0
3	15.0	1208.0	1203.0	33.0	33.0
4	8.0	1557.0	1537.0	42.0	42.0

	Count	Share
Cluster		
0	100	20.0%
1	80	16.0%
2	70	14.0%
3	151	30.2%
4	99	19.8%

🔍 VIP segment validation (high income + high spend):

Top 20% income threshold: 76392

Top 20% spend threshold: 27680

VIP cluster (Cluster 0) income meets threshold: True

VIP cluster (Cluster 0) spend meets threshold: True

🔍 Young segment activity validation:

Youngest cluster (Cluster 1): 25 years old, 6.0 purchases/year

Oldest cluster (Cluster 2): 46 years old, 5.0 purchases/year

Frequency ratio (young/old): 1.20 (≥1.5 aligns with business common sense)

🔍 High-income low-spend segment size validation:

Target clusters: [4]

Total share: 19.8% (15%-30% is reasonable range)

🔍 Clustering vs. manual segmentation match rate: 75.6% (≥70% is considered reliable)

=====

💰 Part 4: CLV Calculation & Result Validation

=====

📌 Customer Lifetime Value (CLV) by segment:

	Avg_CLV	Median_CLV
Cluster		
0	107014.0	104018.0
1	18271.0	17775.0
2	69171.0	68340.0
3	54522.0	53760.0
4	37058.0	37344.0

📋 Sample records for manual verification (copy to Excel to cross-check):

	CustomerID	Average_Order_Value	Purchase_Frequency	CLV
361	C0293	1499	21	94437
73	C0291	1195	16	57360
374	C0289	989	15	44505
155	C0358	868	5	13020
104	C0333	1017	18	54918

⚠️ Outlier check:

Overall CLV mean: 57813
Negative CLV records: 1
Null CLV records: 0
Extreme values (>5× mean): 0 records, 0.0%

=====

🎉 Full pipeline completed! Exported files:

- 1. customer_data.csv — Raw customer data
 - 2. customer_segmentation_with_CLV.csv — Detailed data with cluster labels and CLV
 - 3. segment_profile_summary.csv — Core metrics summary by segment
- =====

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(